

#### ภาคผนวก ค

ข้อรู้สั้ค้ดคลาสการทํางานของ Back – Propagation neural network

```

#include <iostream>
#include <stdlib.h>
#include <time.h>
#include <math.h>           //For tanh

// returns a float in the range -1.0f -> - 1.0f
#define RANDOM_CLAMP      (((float)rand()-(float)rand())/RAND_MAX)

//returns a float between 0 & 1
#define RANDOM_NUM        ((float)rand()/(RAND_MAX+1))

using namespace std;

//*****
class Dendrite {
public:
    double d_weight;           //Weight of the neuron
    unsigned long d_points_to;

    Dendrite(double weight=0.0, unsigned long points_to=0){
        //Constructor
        d_weight = weight;
        d_points_to = points_to;
        //Give it a initial value
    }
};
//*****
class Neuron {
public:
    unsigned long n_ID; //ID of a particular neuron in a layer
    double n_value;     //Value which Neuron is holding
    double n_bias;      //Bias of the neuron
    double n_delta;     //Used in back prop
    Dendrite *Dendrites; //Dendrites
    //Constructor assigning initial values
    Neuron(unsigned long ID=0, double value=0.0, double bias=0.0){
        n_ID = ID;
        n_value = value;
        n_bias = bias;
        n_delta = 0.0;
    }

    void SetDendrites(unsigned long dendrite){
        Dendrites=new Dendrite[dendrite];

        for(int i=0;i<dendrite;i++){ // Initialize the dendrite
            Dendrites[i].d_points_to=i;
        }
    }
};
//*****
class Layer{
public:
    Neuron *Neurons;           //Pointer to array of neurons

    void Initialize(unsigned long size) {
        Neurons = new Neuron [size];
    }
    ~Layer(){
        delete Neurons;
    }
}

```

```

        Neuron GetNeuron(unsigned long index){//Give the neuron
            return Neurons[index];
        }

        void SetNeuron(Neuron neuron,unsigned long index){
            Neurons[index]=neuron;
        }
};
//***** The real neural network *****
class Network {
public:
    double net_learning_rate;           //Learning rate of network
    Layer *Layers;                       //The layers in network
    unsigned long net_tot_layers;        //Number of layers
    double *net_inputs;                 //Input array
    double *net_outputs;                //Output layers
    unsigned long *net_layers;          //Array which tells no. of
                                        //neurons in each layer

    Network() {}
    //----- Load Weight -----
    void LoadWBij (void) {
        .....
    }
    //-----
    void LoadWBjk (void) {
        .....
    }
    //-----
    void LoadBiasJ (void) {
        .....
    }
    //-----
    void LoadBiasK (void) {
        .....
    }
    //*****v
    void InitWB(void){                  //Randomize weights and biases
        int i,j,k;

        for(i=0;i<net_tot_layers;i++){
            for(j=0;j<net_layers[i];j++){ //Last layer does not
                if(i!=(net_tot_layers-1)){ //require weights
                    Layers[i].Neurons[j].SetDendrites(net_layers[i+1]);

                    for(k=0;k<net_layers[i+1];k++){
                        Layers[i].Neurons[j].Dendrites[k].d_weight=GetRand();
                    }
                }
                if(i!=0){                //First layer does not need biases
                    Layers[i].Neurons[j].n_bias=GetRand();
                }
            }
        }
    }
};
//*****

```

```

//----- Record file -----
void ReccordWBij(void){
    .....
}
//*****
void ReccordWBjk(void){
    .....
}
//*****
void ReccordBiasJ(void){
    .....
}
//*****
void ReccordBiasK(void){
    .....
}
//*****
int SetData(double learning_rate,unsigned long layers[],unsigned
            long tot_layers) {//Function to set various
                               //parameters of the net
    if (tot_layers<2) return(-1);//if total no. of layers < 2

    net_learning_rate=learning_rate;
    net_layers= new unsigned long [tot_layers];
    Layers=new Layer[tot_layers];

    for(int i=0;i<tot_layers;i++){
        net_layers[i]=layers[i];
        Layers[i].Initialize(layers[i]);
    }

    net_inputs=new double[layers[0]];
    net_outputs=new double[layers[tot_layers-1]];
    net_tot_layers=tot_layers;

    return 0;
}
//----- Load Input to neurons -----
int SetInputs(double inputs[]){ //Function to set the inputs
    for(int i=0;i<net_layers[0];i++){
        Layers[0].Neurons[i].n_value=inputs[i];
    }
    return 0;
}
//----- Return output to compare -----
double * GetOutput(void){ //Gives the output of the net
    double *outputs;
    int i,j,k;
    outputs=new double[net_layers[net_tot_layers-1]];

    for(i=1;i<net_tot_layers;i++){
        for(j=0;j<net_layers[i];j++){
            Layers[i].Neurons[j].n_value=0;
            for(k=0;k<net_layers[i-1];k++){
                Layers[i].Neurons[j].n_value=Layers[i].Neurons[j].
                    n_value+Layers[i-1].Neurons[k].n_value
                    *Layers[i-1].Neurons[k].Dendrites[j].d_weight;
            }
            Layers[i].Neurons[j].n_value=Layers[i].Neurons[j].
                n_value+Layers[i].Neurons[j].n_bias; //Add bias

```

```

        Layers[i].Neurons[j].n_value=Limiter(Layers[i].
            Neurons[j].n_value);    //Squash that value
    }
}
for(i=0;i<net_layers[net_tot_layers-1];i++){
    outputs[i]=Layers[net_tot_layers-1].Neurons[i].n_value;
}
return outputs;                    //Return the outputs
}

void Update(void){                  //Just a dummy function
    GetOutput();
}
//-----
double ChkStop (double target[]){
    double Target, Actual, Delta;
    Target=target[0];                //Target value
    Actual=Layers[2].Neurons[0].n_value;    //Actual value
    Delta= abs(Target - Actual);
    return Delta;
}
//-----
double Limiter(double value){       //Limiet to limit value
    //between 1 and -1 (sigmoid function)
    return (1.0/(1+exp(-value)));
}
//-----
double GetRand(void){
    time_t timer;
    struct tm *tblock;
    timer=time(NULL);
    tblock=localtime(&timer);

    int seed=int(tblock->tm_sec+100*RANDOM_CLAMP+100*RANDOM_NUM);

    srand(seed);
    return ((RANDOM_CLAMP+RANDOM_NUM)/286);
}
//-----
double SigmaWeightDelta(unsigned long layer_no,
    unsigned long neuron_no){
    double result=0.0;
    for(int i=0;i<net_layers[layer_no+1];i++) {
        result=result+Layers[layer_no].
            Neurons[neuron_no].Dendrites[i].d_weight
            *Layers[layer_no+1].Neurons[i].n_delta;
    }
    return result;
}
//-----
int Train(double inputs[],double outputs[]){
    //The standard Backprop Learning algorithm
    int i,j,k;
    double Target, Actual, Delta, Err;
    Err = 0;
    SetInputs(inputs);                //Set the inputs
    Update();                          //Update all the values
}

```

```

for(i=net_tot_layers-1;i>0;i
  for(j=0;j<net_layers[i];j++) {      //Go thru every neuron
    if(i==net_tot_layers-1){          /** in Output layer
      Target=outputs[j];              //Target value
      Actual=Layers[i].Neurons[j].n_value; //Actual value
      Delta= (Target - Actual) * Actual * (1 - Actual);
      Layers[i].Neurons[j].n_delta=Delta;
      for(k=0;k<net_layers[i-1];k++) {
        Layers[i-1].Neurons[k].Dendrites[j].d_weight +=
          Delta*net_learning_rate*\
          Layers[i-1].Neurons[k].n_value;
      }
      Layers[i].Neurons[j].n_bias =
        Layers[i].Neurons[j].n_bias
        + Delta*net_learning_rate*1;

      Err += ((Target - Actual)*(Target - Actual));

    }
    else {                            //In Hidden layer
      Actual=Layers[i].Neurons[j].n_value; //Actual value
      Delta= Actual * (1 - Actual)* SigmaWeightDelta(i,j);
      for(k=0;k<net_layers[i-1];k++){
        Layers[i-1].Neurons[k].Dendrites[j].d_weight +=
          Delta*net_learning_rate*
          Layers[i-1].Neurons[k].n_value;
      }
      if(i!=0)                        //Input layer does not have a bias
        Layers[i].Neurons[j].n_bias =
          Layers[i].Neurons[j].n_bias+
          Delta*net_learning_rate*1;
    }
  }
}
return Err;
}
// ~Network(){ delete Layers; }
};

//*****
//*****

void main (void){

  Network my;

  unsigned long inp=646;
  unsigned long hid=20;
  unsigned long outp=10;
  unsigned long layers[3];

  layers[0]=inp;
  layers[1]=hid;
  layers[2]=outp;

  int i=0,j=0,k=0,count=0;

  double max;
  int pnt;

  double ERR,SQERR;

```

```

    //double input[]={1,0};
    double *outputs;

unsigned long iter=0;

    my.SetData(0.2, layers,3);

    my.InitWB();

double tr_inp[200][646];

    double buff[38][17];
    double test[1][646];

    double cmd[10][10] = {
{1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0},
{0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0},
{0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0},
{0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0},
{0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0},
{0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0},
{0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0},
{0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0},
{0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0},
{0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0}};

    char ch;

    //*****
    for(i=0 ; i<1 ; i++){
        char* aa;
        string axx, str;
        aa = "tr_input.txt";

        string constructorString (myread(aa));

        int x = 0;
        int y = 0;

        axx = myread("tr_input.txt");

        const char* str2;

        j=0;
        k=0;

        for(int s = 0; s < filesize("tr_input.txt"); s++){

            if ( constructorString[s] != '\t' &&
constructorString[s] != '\n' ){x++;}

```

```

else {
    if (constructorString[s] == '\n')
        x = x-1;

    str = axx.substr(y,x);
    str2= str.c_str();

// const char

    float tempMatrix = atof(str2);

    tr_inp[j][k] = tempMatrix;

    y = y+x+1;
    x = 0;

    k++;

    if(k==646)
    {
        j++;
        k=0;
    };

    if (constructorString[s] == '\n')
        y = y+1;
    }
}

//*****
//*****
for(i=0 ; i<1 ; i++){
    char* aa;
    string axx,str;
    aa = "Ceptrum0.txt";

    string constructorString (myread(aa));

    int x = 0;
    int y = 0;

    axx = myread("Ceptrum0.txt");

    const char* str2;

    int j=0,k=0;

    for(int s = 0; s < filesize("Ceptrum0.txt"); s++){

        if ( constructorString[s] != '\t' &&
constructorString[s] != '\n' ){x++;}
        else {
            if (constructorString[s] == '\n')

                x = x-1;

```



```

        str  = axx.substr(y,x);
        str2= str.c_str();

// const char

        float      tempMatrix = atof(str2);

        buff[j][k]  = tempMatrix;

        y = y+x+1;
        x = 0;

        k++;

        if(k==17)

        {
                j++;
                k=0;
        };

        if (constructorString[s] == '\n')

                y = y+1;

        }

    }

    for(i=0;i < 38;i ++){
        for(j=0;j<17;j++){
            test[0][k] = buff[i][j];
            k++;
        }
    }
}
//*****

cout << "<L>    for Loading database \n";
cout << "<R>    for Recognition \n";
cout << "<T>    for Training \n";
cout << "<Q>    for Quit program \n";                                cout <<
"Press only 1 choice then Enter : ";
cin >> ch;

while (ch != 'q' && ch != 'Q'){

//----- LOAD WEIGHT -----
-----

    if( ch == 'l' || ch == 'L'){
        cout << "\nLoading database... \n";

        my.LoadWBij();
        my.LoadWBjk();
        my.LoadBiasJ();
        my.LoadBiaskK();

        cout << "Ending Loading";
    }
}

```

```

        cout << "\n\n";
    }

    //----- TRAINING -----
    -----

    if( ch == 't' || ch == 'T'){
        cout <<"Enter number of training Iterations : ";
        cin >>iter;
        cout<<"\nStarting Training... ";
        for(i=0;i<iter;i++){
            k = 0;
            count = 0;

            //cout<<"\nTraining : "<<i+1;

            for(j=0;j<200;j++){
                ERR = my.Train(tr_inp[j],cmd[count]);
                count++;

                if(count > 9 ){
                    // k ++;
                    count = 0;
                }

                SQQERR += ERR;
            }

            k++;

            SQQERR = SQQERR/2;

            cout << count << " -> SQQERR = " << SQQERR
<<"\n";

        }

        //*****
        my.ReccordWBij();
        my.ReccordWBjk();
        my.ReccordBiasJ();
        my.ReccordBiaskK();
        //*****

        cout<<"\nEnding Training. ";
    }

    //----- RECOGNIZE -----
    -----

    if( ch == 'r' || ch == 'R'){

```

```

        cout<<"\n\nStarting Testing... \n";
        for(j=0;j<1;j++){
            my.SetInputs(test[j]);
            outputs=my.GetOutput();
            for(i=0;i<outp;i++){
                cout<<"\nOutput"<<i<<" :
";
                cout<<outputs[i];
            }
            for(i=0;i<outp;i++){
                if (outputs[i] > max){
                    max = outputs[i];
                    pnt = i;
                }
            }
            if (pnt == 0) cout <<"\n\n You say : word ";
            if (pnt == 1) cout <<"\n\n You say :
Powerpoint ";
            if (pnt == 2) cout <<"\n\n You say : Excel
";
            if (pnt == 3) cout <<"\n\n You say : NotePad
";
            if (pnt == 4) cout <<"\n\n You say : Paint
";
            if (pnt == 5) cout <<"\n\n You say :
Internet ";
            if (pnt == 6) cout <<"\n\n You say : E-mail
";
            if (pnt == 7) cout <<"\n\n You say : MSN ";
            if (pnt == 8) cout <<"\n\n You say : Music
";
            if (pnt == 9) cout <<"\n\n You say : Pinball
";

            max = 0.0;

            delete outputs;

            double *outputs;
        }

        cout<<"\n\nEnd Testing.\n\n";
    }

    cout << "\n-----\n\n";
    cout << "<L>    for Loading database \n";
    cout << "<R>    for Recognition \n";
    cout << "<T>    for Training \n";
    cout << "<Q>    for Quit program \n\n";
    cout
<< "Press only 1 choice then Enter : ";
    cin >> ch;
    cout << "\n";

```

```

        }
        cin.get();
    }

//*****
*****

//***** Support File
*****

char * myread(char *filename)
{
    FILE *fp;

    fp = fopen(filename, "rb");
    fseek(fp, 0L, SEEK_END );
    size_t endPos = ftell( fp );

    fseek(fp, 0, SEEK_SET);

    char *tag_buffer;
    tag_buffer = (char*) malloc (endPos);

    fread(tag_buffer, sizeof(char),endPos, fp);
    fclose(fp);
    return tag_buffer;
}

int filesize(char *filename)
{
    FILE *fp;

    fp = fopen(filename, "rb");
    fseek(fp, 0L, SEEK_END );
    size_t endPos = ftell( fp );
    fclose(fp);
    return endPos;
}

//*****
*****

```